

918. Maximum Sum Circular Subarray

Medium Topics Companies Hint

Given a **circular integer array** `nums` of length `n`, return the *maximum possible sum of a non-empty subarray* of `nums`.

A **circular array** means the end of the array connects to the beginning of the array. Formally, the next element of `nums[i]` is `nums[(i + 1) % n]` and the previous element of `nums[i]` is `nums[(i - 1 + n) % n]`.

A **subarray** may only include each element of the fixed buffer `nums` at most once. Formally, for a subarray `nums[i], nums[i + 1], ..., nums[j]`, there does not exist `i ≤ k1, k2 ≤ j` with `k1 % n == k2 % n`.

Example 1:

Input: `nums = [1,-2,3,-2]`

Output: 3

Explanation: Subarray `[3]` has maximum sum 3.

Example 2:

Input: `nums = [5,-3,5]`

Output: 10

Explanation: Subarray `[5,5]` has maximum sum `5 + 5 = 10`.

Example 3:

Input: `nums = [-3,-2,-3]`

Output: -2

Explanation: Subarray `[-2]` has maximum sum -2.

Constraints:

- `n == nums.length`
- `1 ≤ n ≤ 3 * 104`
- `-3 * 104 ≤ nums[i] ≤ 3 * 104`

7.5K 141 ☆

35 Online

```
1 class Solution {
2     public int maxSubarraySumCircular(int[] nums) {
3
4         int totalSum = 0;
5
6         int currentMax = nums[0];
7         int maxSum = nums[0];
8
9         int currentMin = nums[0];
10        int minSum = nums[0];
11
12        totalSum += nums[0];
13
14        for (int i = 1; i < nums.length; i++) {
15
16            // Maximum subarray sum
17            currentMax = Math.max(nums[i], currentMax + nums[i]);
18            maxSum = Math.max(maxSum, currentMax);
19
20            // Minimum subarray sum
21            currentMin = Math.min(nums[i], currentMin + nums[i]);
22            minSum = Math.min(minSum, currentMin);
23
24            totalSum += nums[i];
25        }
26
27        // If all elements are negative
28        if (maxSum < 0) {
29            return maxSum;
30        }
31
32        // Maximum circular sum
33        int circularSum = totalSum - minSum;
34
35        return Math.max(maxSum, circularSum);
36    }
37 }
```

Saved

Ln 37, Col 2

Testcase Test Result

918. Maximum Sum Circular Subarray

Medium Topics Companies Hint

Given a **circular integer array** `nums` of length `n`, return the *maximum possible sum of a non-empty subarray* of `nums`.

A **circular array** means the end of the array connects to the beginning of the array. Formally, the next element of `nums[i]` is `nums[(i + 1) % n]` and the previous element of `nums[i]` is `nums[(i - 1 + n) % n]`.

A **subarray** may only include each element of the fixed buifer `nums` at most once. Formally, for a subarray `nums[i], nums[i + 1], ..., nums[j]`, there does not exist $1 \leq k1, k2 \leq j$ with $k1 \% n == k2 \% n$.

Example 1:

Input: `nums = [1,-2,3,-2]`
Output: 3
Explanation: Subarray [3] has maximum sum 3.

Example 2:

Input: `nums = [5,-3,5]`
Output: 10
Explanation: Subarray [5,5] has maximum sum 5 + 5 = 10.

Example 3:

Input: `nums = [-3,-2,-3]`
Output: -2
Explanation: Subarray [-2] has maximum sum -2.

Constraints:

- `n == nums.length`
- $1 \leq n \leq 3 * 10^4$
- $-3 * 10^4 \leq nums[i] \leq 3 * 10^4$

Code

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

`nums =`
`[1,-2,3,-2]`

Output

3

Expected

3

♥ Contribute a testcase